- **用类来描述实例的类型**

```
class Person {}
const person: Person = new Person()
```

- **所有的实例上的属性都需要先声明再使用**

```
class Circle{
  x:number
  y:number
  constructor(x: number, y: number = 100, ...args: number[]){
    this.x = x
    this.y = y
  }
}
```

- **类的修饰符**
  - 修饰符种类
    - public（默认值）
      - 公开的：我自己、我的儿子、外界

```
class Animal {
  public name
  public age
  constructor(name: string, age: number){
    this.name = name
    this.age = age
  }
}
// 或
class Animal {
  // 直接在参数中添加public、protected、private修饰符，这些属性会默认添加到实例
  constructor(public name: string, public age: number){
  }
}
```

```
class Cat extends Animal {
  constructor(name: string, age: number){
    super(name, age)
    this.name = 'Orange' // 孩子可读写
  }
}
let cat = new Cat('Tom', 10)
console.log(cat.name, cat.age) // 外界可读写
```

- protected
  - 受保护的：我自己、我的儿子

```
class Animal {
  // 直接在参数中添加public、protected、private修饰符，这些属性会默认添加到实例
  constructor(protected name: string, public age: number){
  }
}

class Cat extends Animal {
  constructor(name: string, age: number){
    super(name, age)
    this.name = 'Orange' // 孩子可读写
  }
}
let cat = new Cat('Tom', 10)
console.log(cat.name, cat.age) // name报错：属性“name”受保护，只能在类“Animal”及其子类中访问。
```

- private
  - 私有的

```
class Animal {
  // 直接在参数中添加public、protected、private修饰符，这些属性会默认添加到实例
  constructor(private name: string, public age: number){
  }
}
class Cat extends Animal {
  constructor(name: string, age: number){
```

```
    super(name, age)
    this.name = 'Orange' // 报错：属性“name”为私有属性，只能在类“Animal”中访问。
  }
}

let cat = new Cat('Tom', 10)
console.log(cat.name, cat.age) // name报错：属性“name”为私有属性，只能在类“Animal”中访问。
```

- readonly
  - 只读的
  - 不与public、protected、private冲突

```
class Animal {
  // 直接在参数中添加public、protected、private修饰符，这些属性会默认添加到实例
  constructor(public readonly name: string, public age: number){
    this.name = 'White' // 不会报错，因为构造函数是初始化阶段
  }
  changeName(){
    this.name = 'Black' // 报错：无法为“name”赋值，因为它是只读属性。
  }
}
class Cat extends Animal {
  constructor(name: string, age: number){
    super(name, age)
    this.name = 'Orange' // 报错：无法为“name”赋值，因为它是只读属性。
  }
}
```

- 修饰符在构造函数上的用法
  - protected
    - 父类可以被继承，但不能被实例化
    - 类似抽象类

```
class Animal {
  protected constructor(){}
}
class Mouse extends Animal {
```

```
  constructor(){
    super()
  }
}

// new Animal() // 报错：类“Animal”的构造函数是受保护的，仅可在类声明中访问。
new Mouse()
```

- private
  - 无法被继承或实例化

```
class Animal {
  private constructor(){}
}

// class Mouse extends Animal {} // 报错：无法扩展类“Animal”。类构造函数标记为私有。
// new Animal() // 报错：类“Animal”的构造函数是私有的，仅可在类声明中访问。
```

  - 应用场景：单例模式

```
class Singleton {
  private constructor(){}
  static instance = new this()
  static getInstance(){
    return this.instance
  }
}

let instance1 = Singleton.getInstance()
let instance2 = Singleton.getInstance()
console.log(instance1 === instance2) // true
```

- **子类中重写父类的方法，当父类的返回值不是void时，子类的返回值类型必须和父类一致**

```
class Animal {
```

```
  say(): string {
    return '1'
  }
}
class Mouse extends Animal {
  say(): number { // 报错：类型“Mouse”中的属性“say”不可分配给基类型“Animal”中的同一属性。
    return 1
  }
}

// ----------------------------

class Animal {
  say(): void {}
}
class Mouse extends Animal {
  say(): number { // 允许，因为父类不关心返回值
    return 1
  }
}
```

- **抽象**
  - 抽象类可以被继承，但不能被实例化

```
abstract class Person {}
class Teacher extends Person {}
new Person() // 报错：无法创建抽象类的实例。
```

  - 抽象方法只能写在抽象类中

```
abstract class Person {
  abstract eat(): void // 非抽象类会报错：抽象方法只能出现在抽象类中。
}
```

  - 父类定义的抽象方法，非抽象子类必须重写

```
abstract class Person {
  abstract eat(): void
}
class Teacher extends Person {
  eat(){} // 不写会报错：非抽象类“Teacher”不会实现继承自“Person”类的抽象成员“eat”。
}
```